

Documento Técnico de Desenvolvimento do Jogo Stealth 3D na Unity

1. Introdução

O presente documento detalha o processo de desenvolvimento do jogo “Stealth 3D”, concebido na plataforma Unity. O projeto teve como objetivo principal a criação de um ambiente interativo onde o jogador controla um personagem com a tarefa de infiltrar-se em uma base inimiga, evitando guardas e sistemas de detecção. As mecânicas centrais do jogo incluem movimentação em terceira pessoa, patrulhamento de guardas com campo de visão (cone de detecção) e um sistema de alerta. A escolha por este conceito foi motivada pela complexidade e pelo potencial de aplicação de inteligência artificial (IA) em um cenário de infiltração, proporcionando uma experiência de jogo estratégica e imersiva.

2. Planejamento e Ferramentas

2.1 Concepção do Projeto e Divisão de Tarefas

Inicialmente, a equipe considerou duas propostas de jogo: “Stealth Game” e “Drone Delivery”. Após análise, optou-se pelo “Stealth Game” devido ao seu maior potencial de engajamento e à oportunidade de explorar a IA de forma mais aprofundada. A equipe foi então organizada com responsabilidades específicas para cada membro:

- Caio:** Desenvolvimento do menu inicial, tela de game over, créditos e opções.
- Brian:** Criação dos bots (inimigos) e implementação de todas as mecânicas de IA relacionadas a eles.
- Rafael:** Construção e configuração do cenário completo.
- Felipe:** Implementação da movimentação do personagem principal e mecânicas associadas.

2.2 Ferramentas de Desenvolvimento

O desenvolvimento do projeto foi realizado utilizando a **Unity Editor** como ambiente principal. Para o controle de versão e colaboração, foi empregado o **GitHub**, onde o repositório do projeto foi criado e compartilhado entre os membros da equipe. A codificação foi majoritariamente feita no **VS Code**. Para auxiliar na resolução de problemas e na geração de trechos de código, foram utilizadas ferramentas de inteligência artificial como **ChatGPT** e **Claude**. A IA “Manus” também foi consultada para orientações específicas, como na configuração de iluminação.

2.3 Desafios Iniciais e Soluções Adotadas

Durante a fase de configuração do ambiente de desenvolvimento, foi identificado um problema significativo na integração da extensão “Continue” com o “Ollama” no VS Code. Este problema manifestou-se como um estado de “Loading config” infinito e a ausência de modelos configurados, indicando uma falha na detecção ou carregamento do arquivo de configuração. As tentativas de solução incluíram a recriação manual do arquivo `config.json`, a reinicialização do VS Code, a verificação do modelo no Ollama e a configuração manual do provedor. Contudo, essas abordagens não foram eficazes. A solução adotada foi descontinuar o uso da extensão “Continue” e do “Ollama”, optando por utilizar o Claude e o ChatGPT como ferramentas principais de assistência, o que permitiu o prosseguimento do desenvolvimento sem bloqueios.

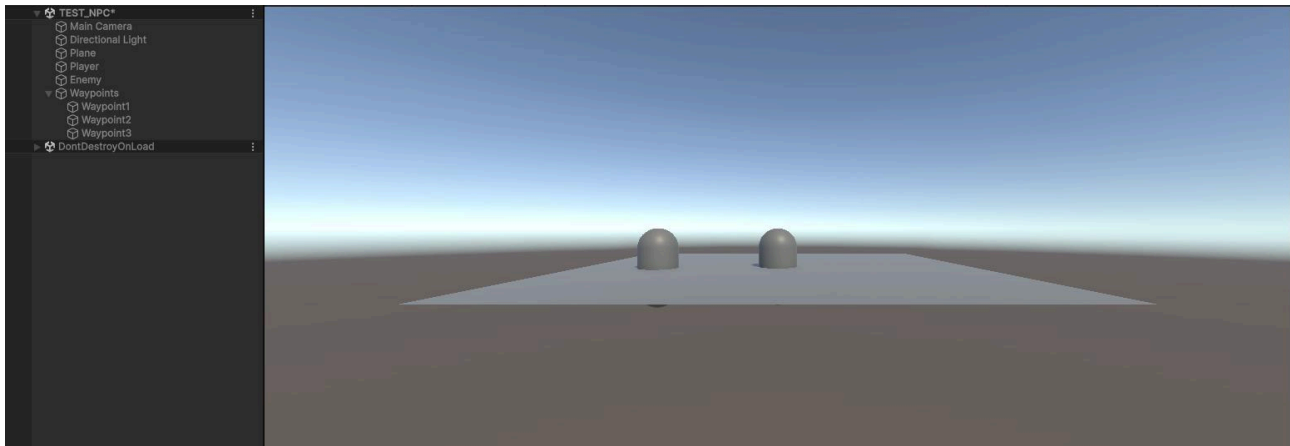
3. Desenvolvimento Técnico

3.1 Interface e Menus

A implementação da interface do usuário (UI) e dos menus foi uma das primeiras etapas do desenvolvimento. O processo envolveu a criação de um `canvas` na Unity, que serve como a área onde todos os elementos da UI são renderizados. Botões para “Iniciar”, “Opções” e “Créditos” foram adicionados, e painéis correspondentes foram criados para cada seção do menu. A interatividade dos botões, como a mudança de cor ao passar o mouse, foi configurada para melhorar a experiência do usuário. O script `MenuController.cs` foi desenvolvido para gerenciar a visibilidade dos painéis e a navegação entre as diferentes seções do menu. Abaixo, apresenta-se o código-fonte do `MenuController.cs`:

O código completo do `MenuController.cs` pode ser consultado no Anexo A.1.

Este script permite a ativação e desativação dos painéis de menu, garantindo que apenas a seção relevante esteja visível para o jogador. Os comentários foram inseridos para facilitar a compreensão do código e sua manutenção.



3.2 Cenário e Atmosfera

A criação de um cenário noturno imersivo foi fundamental para o “Stealth 3D”. Esta etapa envolveu a configuração detalhada de iluminação, Skybox, neblina (Fog) e efeitos de pós-processamento. A IA “Manus” foi utilizada como guia para a implementação dessas configurações.

3.2.1 Configuração de Iluminação e Skybox Noturno

A iluminação é um elemento crucial para estabelecer o clima visual do jogo. As configurações foram ajustadas para simular um ambiente escuro, iluminado predominantemente pela lua. O processo seguiu os seguintes passos:

1. Configuração da Iluminação Ambiente:

- No Unity Editor, acessou-se `Window > Rendering > Lighting > Settings`.
- Na janela `Lighting`, localizou-se a seção `Environment`.
- Definiu-se o `Source` de `Environment Lighting` para `Color`.
- Clicou-se na caixa de cor ao lado de `Ambient Color` e escolheu-se um tom de azul escuro ou cinza muito escuro (ex: `#1A2B3C` ou `#0A0A0A`) para simular a luz ambiente noturna.

- Assegurou-se que o `Ambient Intensity` estivesse em um valor baixo (0.1 a 0.3) para manter a escuridão geral.

2. Configuração do Skybox Noturno:

- O `Skybox` define o céu e o ambiente distante. Para um cenário noturno, um `Skybox` escuro com estrelas é ideal.
- **Criação do Material de Skybox:** No painel `Project`, clicou-se com o botão direito do mouse e selecionou-se `Create > Material`. O material foi renomeado para `NightSkybox`. No `Inspector` do `NightSkybox`, o `Shader` foi alterado para `Skybox/Procedural` ou `Skybox/6 Sided`. Se `Skybox/Procedural` foi utilizado, os parâmetros como `Sun Size`, `Atmosphere Thickness` e `Sky Tint` foram ajustados para criar um céu escuro com estrelas visíveis, e o `Exposure` foi definido para um valor baixo.
- **Aplicação do Skybox:** Na janela `Lighting` (`Window > Rendering > Lighting > Settings`), na seção `Environment`, arrastou-se o material `NightSkybox` para o slot `Skybox Material`.

3. Ajuste da Luz Direcional (Lua):

- Uma luz direcional foi configurada para simular a luz da lua, criando sombras longas e dramáticas.
- No painel `Hierarchy`, selecionou-se o objeto `Directional Light` (se não houvesse, um foi criado clicando com o botão direito do mouse em `Hierarchy > Light > Directional Light`).
- No `Inspector` do `Directional Light`, a `Color` foi alterada para um tom azulado claro ou branco acinzentado (ex: `#A0B0C0` ou `#D0E0F0`).
- A `Intensity` foi ajustada entre 0.5 e 1.0, dependendo da intensidade desejada da luz da lua.
- O `Directional Light` foi girado para que as sombras projetadas criassem o efeito desejado (geralmente vindo de um ângulo baixo).
- Assegurou-se que o `Shadow Type` estivesse definido como `Soft Shadows` ou `Hard Shadows` para sombras realistas.

3.2.2 Configuração de Neblina e Atmosfera (Fog)

A neblina é essencial para adicionar profundidade e mistério ao cenário noturno, simulando a umidade do ar e limitando a visibilidade. Os passos para sua configuração foram:

1. Ativação e Configuração da Neblina Global:

- No Unity Editor, acessou-se `Window > Rendering > Lighting > Settings`.
- Na janela `Lighting`, localizou-se a seção `Other Settings`.
- Marcou-se a caixa `Fog` para ativá-la.
- Definiu-se o `Mode` para `Exponential Squared` para uma neblina mais densa e realista.
- Clicou-se na caixa de cor ao lado de `Fog Color` e escolheu-se um tom de azul escuro ou cinza muito escuro (ex: `#101520` ou `#050505`), semelhante à cor ambiente.
- Ajustou-se o `Density` da neblina, começando com um valor baixo (0.01 a 0.05) e aumentando gradualmente.

2. Considerações Adicionais:

- A `Distância de Renderização da Câmera (Far Clipping Plane)` da câmera principal foi verificada para garantir que a neblina fosse visível e eficaz.
- A `Performance` foi considerada, com a sugestão de usar um `Mode` de neblina mais simples (`Linear`) ou valores menores de `Density` em caso de impacto no desempenho.

3.2.3 Configuração de Pós-Processamento (Volume)

Os efeitos de pós-processamento foram aplicados para refinar a atmosfera visual, ajustando cores, contraste e adicionando efeitos como bloom e vinheta. O procedimento foi o seguinte:

1. Instalação do Pacote Post Processing:

- No Unity Editor, acessou-se `Window > Package Manager`.

- No menu suspenso, selecionou-se `Unity Registry`.
- Procurou-se por `Post Processing` e clicou-se em `Install`.

2. Criação de um Volume de Pós-Processamento:

- No painel `Hierarchy`, clicou-se com o botão direito do mouse e selecionou-se `3D Object > Volume > Global Volume`.
- O objeto foi renomeado para `NightForestPostProcessVolume`.
- No `Inspector` do `NightForestPostProcessVolume`, marcou-se a caixa `Is Global`.
- Clicou-se em `New` ao lado de `Profile` para criar um novo perfil de pós-processamento.

3. Adição de Efeitos de Pós-Processamento:

- Com o perfil criado, os seguintes efeitos foram adicionados e ajustados para o cenário noturno:
 - **Bloom:** Adicionado para um brilho suave em áreas claras, ajustando `Intensity` e `Threshold`.
 - **Color Grading:** Utilizado para ajustar as cores gerais da cena, adicionando um tom azulado frio e aumentando o contraste. No modo HDR, `Temperature` e `Tint` foram ajustados, assim como `Contrast` e `Saturation`.
 - **Vignette:** Escureceu as bordas da tela para focar a atenção no centro, ajustando `Intensity` e `Smoothness`.
 - **Ambient Occlusion (Screen Space Ambient Occlusion - SSAO):** Adicionou sombras sutis para aumentar a profundidade e o realismo, ajustando `Intensity` e `Radius`.
 - **Grain:** Adicionou um ruído sutil à imagem, ajustando `Intensity` e `Size`.

3.3 Movimentação do Jogador

A movimentação do jogador foi implementada para permitir controle intuitivo e dinâmico do personagem. Esta seção detalha a criação do script de controle e a integração da barra de stamina na interface do usuário.

3.3.1 Script de Controle de Movimento e Stamina (`PlayerSettings.cs`)

O script `PlayerSettings.cs` foi desenvolvido para gerenciar a movimentação do personagem (WASD), a corrida com a tecla Shift e um sistema de stamina. A IA “Claude” foi utilizada para auxiliar na escrita do código. O sistema de stamina impede que o personagem corra continuamente, adicionando uma camada estratégica ao gameplay. Abaixo, apresenta-se o código-fonte do `PlayerSettings.cs`:

O código completo do `PlayerSettings.cs` pode ser consultado no Anexo A.2.

3.3.2 Configuração da Interface da Barra de Stamina

Para visualizar a stamina do jogador, uma barra de progresso foi implementada na UI. Os passos para a criação e configuração da barra de stamina foram:

1. **Criação do Canvas:** Um novo `Canvas` foi criado (`Hierarchy` → clique direito → `UI` → `Canvas`) e seu `Render Mode` foi definido como `Screen Space - Overlay`.
2. **Criação do Slider:** Dentro do `Canvas`, um `Slider` foi adicionado (Clique com o botão direito do mouse no `Canvas` → `UI` → `Slider`) e renomeado para `StaminaBar`.
3. **Configuração do Slider:** No `Inspector` do `StaminaBar`, os valores `Value`, `Min Value` e `Max Value` foram definidos como 100, 0 e 100, respectivamente. A opção `Interactable` foi desmarcada e `Transition` foi definido como `None`.
4. **Remoção do Handle:** O objeto `Handle Slide Area` foi removido do `StaminaBar` na `Hierarchy`, pois uma barra de stamina não requer um controle arrastável.
5. **Coloração do Preenchimento:** Dentro de `StaminaBar` → `Fill Area` → `Fill`, a cor do componente `Image` foi alterada para verde (`#36D966`).
6. **Posicionamento da Barra:** No `Rect Transform` do `StaminaBar`, o `Anchor` foi definido como `Top Left`. As posições `Pos X` e `Pos Y` foram ajustadas para 160 e -30, respectivamente, e `width` e `Height` para 300 e 20.
7. **Conexão ao Script:** No objeto `Capsule` (representando o jogador), os campos `Barra Stamina` e `Preenchimento Stamina` do componente `PlayerController` foram preenchidos arrastando-se o objeto `StaminaBar` e seu componente `Fill` (dentro de `Fill Area`), respectivamente.

3.4 Inteligência Artificial dos Inimigos

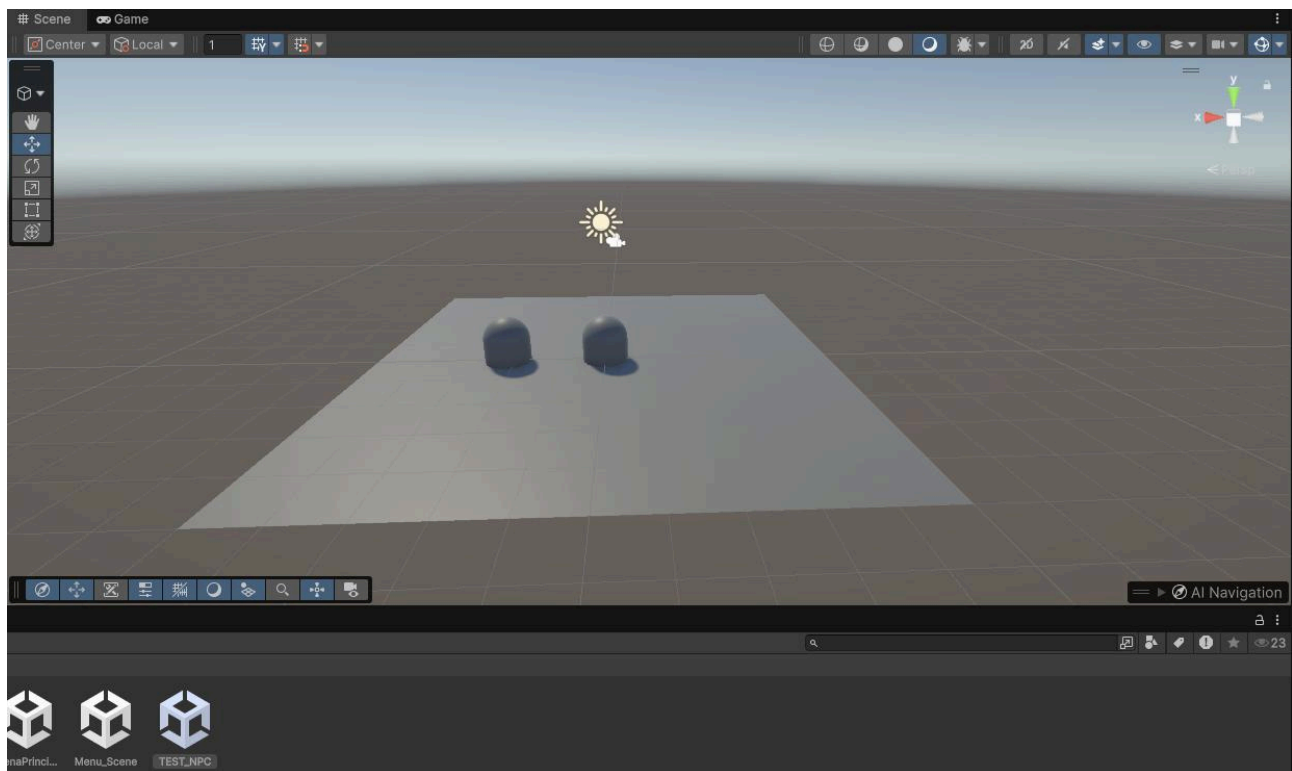
A inteligência artificial dos inimigos (NPCs) foi desenvolvida de forma modular, permitindo um comportamento de patrulha, visão e detecção robusto. A metodologia de desenvolvimento focou na separação da lógica de comportamento da representação visual.

3.4.1 Planejamento da IA e Cena de Teste

Para o desenvolvimento da IA, foi adotada a prática de criar uma cena de teste isolada, denominada `Test_NPC`. Esta abordagem permitiu testar a lógica e o comportamento dos NPCs sem a dependência do cenário final ou dos modelos gráficos. A cena de teste consistiu em:

1. **Chão:** Um objeto `Plane` (`3D Object` → `Plane`) foi criado para servir como superfície.
2. **Jogador (Placeholder):** Um objeto `Capsule` (`3D Object` → `Capsule`) foi criado e renomeado para `Player`, atuando como um substituto temporário para o personagem principal.
3. **NPC (Placeholder):** Outro objeto `Capsule` (`3D Object` → `Capsule`) foi criado e renomeado para `Enemy`, servindo como substituto temporário para o inimigo.

Essa configuração simplificada garantiu que o foco estivesse na lógica da IA, e não em aspectos gráficos ou de animação, seguindo o princípio de separar a lógica do visual. A ordem de desenvolvimento da IA seguiu as etapas de Patrulha, Sistema de Visão e Detecção, com a integração final com o player real e a UI de derrota sendo realizada posteriormente.



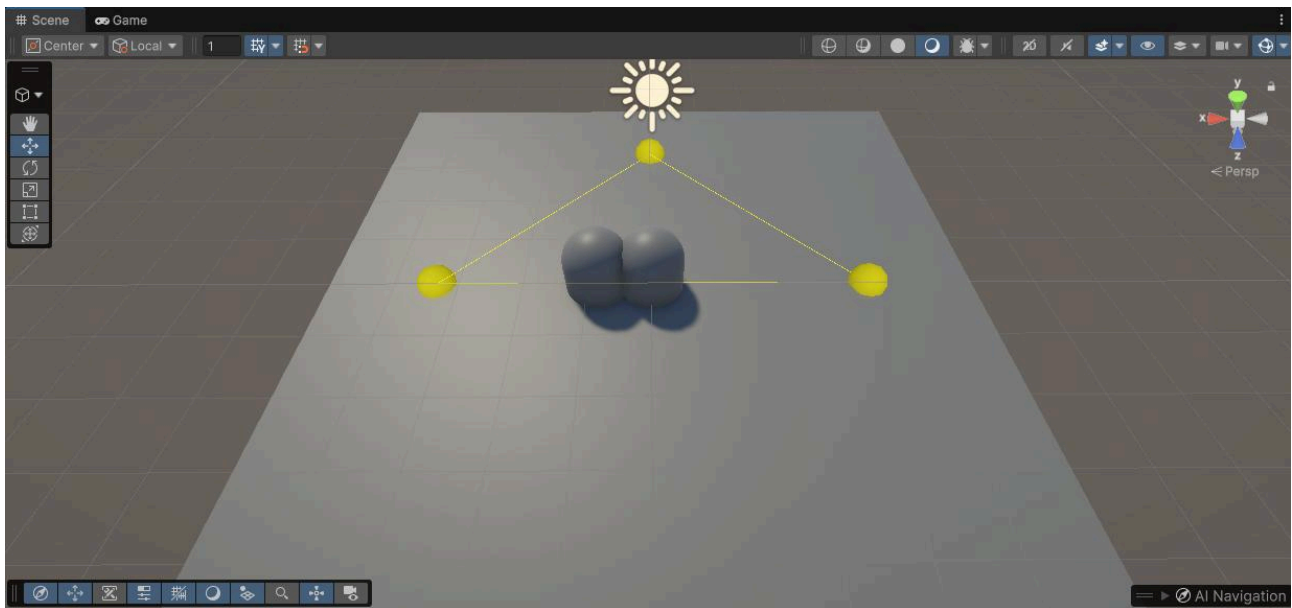
3.4.2 Sistema de Patrulha do NPC (`EnemyPatrol.cs`)

O sistema de patrulha permite que o NPC se mova automaticamente entre pontos predefinidos no cenário. Os passos para sua implementação foram:

1. **Criação de Pontos de Patrulha (Waypoints):** Três objetos vazios (`Hierarchy` → `Create Empty`) foram criados e renomeados para `Waypoint1` , `Waypoint2` e `Waypoint3` . Esses waypoints foram posicionados no chão para definir a rota de patrulha. Para organização, um objeto vazio pai, `Waypoints` , foi criado, e os waypoints individuais foram colocados como seus filhos.
2. **Criação do Script `EnemyPatrol.cs` :** Um novo script C# foi criado com o nome `EnemyPatrol` . O código-fonte é apresentado abaixo:

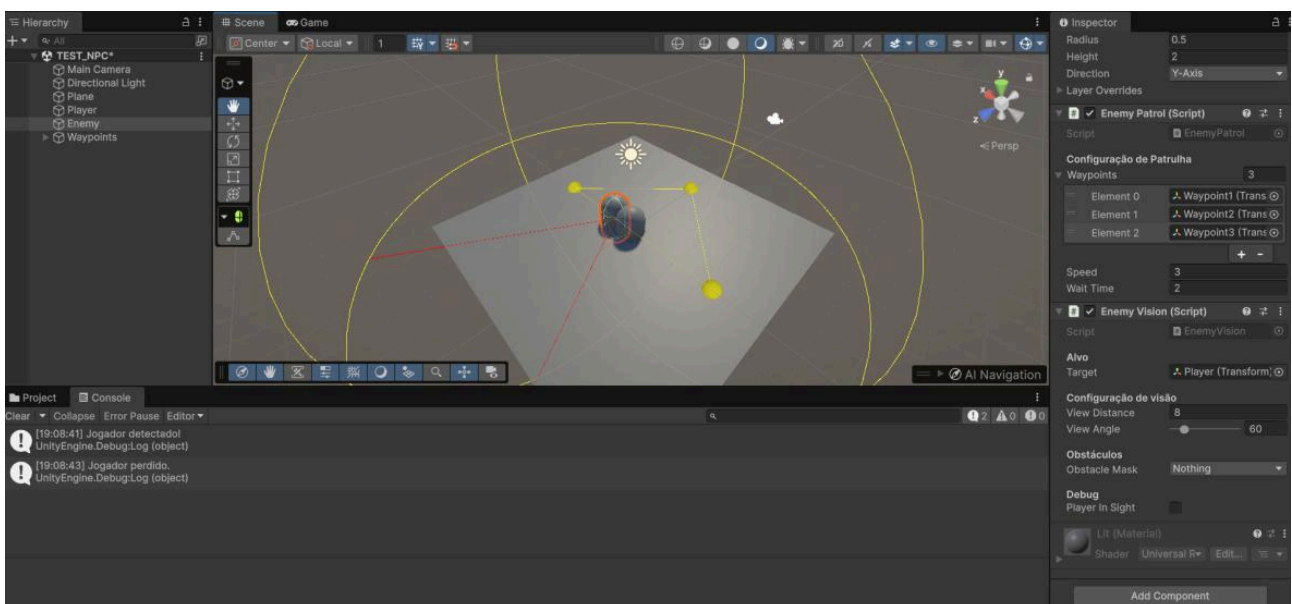
O código completo do `EnemyPatrol.cs` pode ser consultado no Anexo A.3.

1. **Configuração na Unity:** No objeto `Enemy` , o componente `EnemyPatrol` foi adicionado (`Add Component` → `EnemyPatrol`). Os waypoints foram arrastados do painel `Hierarchy` para o array `Waypoints` no `Inspector` do componente `EnemyPatrol` .



3.4.3 Sistema de Visão do NPC (`EnemyVision.cs`)

O sistema de visão do NPC permite que ele detecte o jogador dentro de um cone de visão e identifique obstáculos que possam bloquear essa visão. O script `EnemyVision.cs` foi criado para gerenciar essa funcionalidade. O código completo do `EnemyVision.cs` pode ser consultado no Anexo A.4.



3.4.4 Sistema de Detecção e Reação do NPC (`EnemyDetection.cs`)

O script `EnemyDetection.cs` é responsável por processar as informações do `EnemyVision.cs` e determinar a reação do NPC ao detectar o jogador. Esta separação de responsabilidades entre “ver” e “reagir” torna a IA mais modular e fácil de manter.

Quando o NPC detecta o alvo, ele interrompe a patrulha, gira para olhar para o alvo e entra em estado de alerta. O código-fonte é apresentado abaixo:

```

using UnityEngine;

public class EnemyVision : MonoBehaviour
{
    public float viewDistance = 10f;
    [Range(0, 180)]
    public float viewAngle = 90f;
    public LayerMask targetMask;
    public LayerMask obstacleMask;

    [HideInInspector]
    public Transform target;
    [HideInInspector]
    public bool playerInSight;

    void Update()
    {
        FindTarget();
    }

    void FindTarget()
    {
        playerInSight = false;
        target = null;

        Collider[] targetsInViewRadius =
Physics.OverlapSphere(transform.position, viewDistance, targetMask);

        for (int i = 0; i < targetsInViewRadius.Length; i++)
        {
            Transform potentialTarget = targetsInViewRadius[i].transform;
            Vector3 dirToTarget = (potentialTarget.position -
transform.position).normalized;

            if (Vector3.Angle(transform.forward, dirToTarget) < viewAngle /
2)
            {
                float dstToTarget = Vector3.Distance(transform.position,
potentialTarget.position);

                if (!Physics.Raycast(transform.position, dirToTarget,
dstToTarget, obstacleMask))
                {
                    target = potentialTarget;
                    playerInSight = true;
                }
            }
        }
    }
}

```

```

        break;
    }
}

}

}

void OnDrawGizmos()
{
    Vector3 origin = transform.position;
    Vector3 forward = transform.forward * viewDistance;

    Quaternion leftRotation = Quaternion.Euler(0f, -viewAngle / 2f, 0f);
    Quaternion rightRotation = Quaternion.Euler(0f, viewAngle / 2f, 0f);

    // Cone de visão
    Gizmos.color = Color.red;
    Gizmos.DrawRay(origin, leftRotation * forward);
    Gizmos.DrawRay(origin, rightRotation * forward);

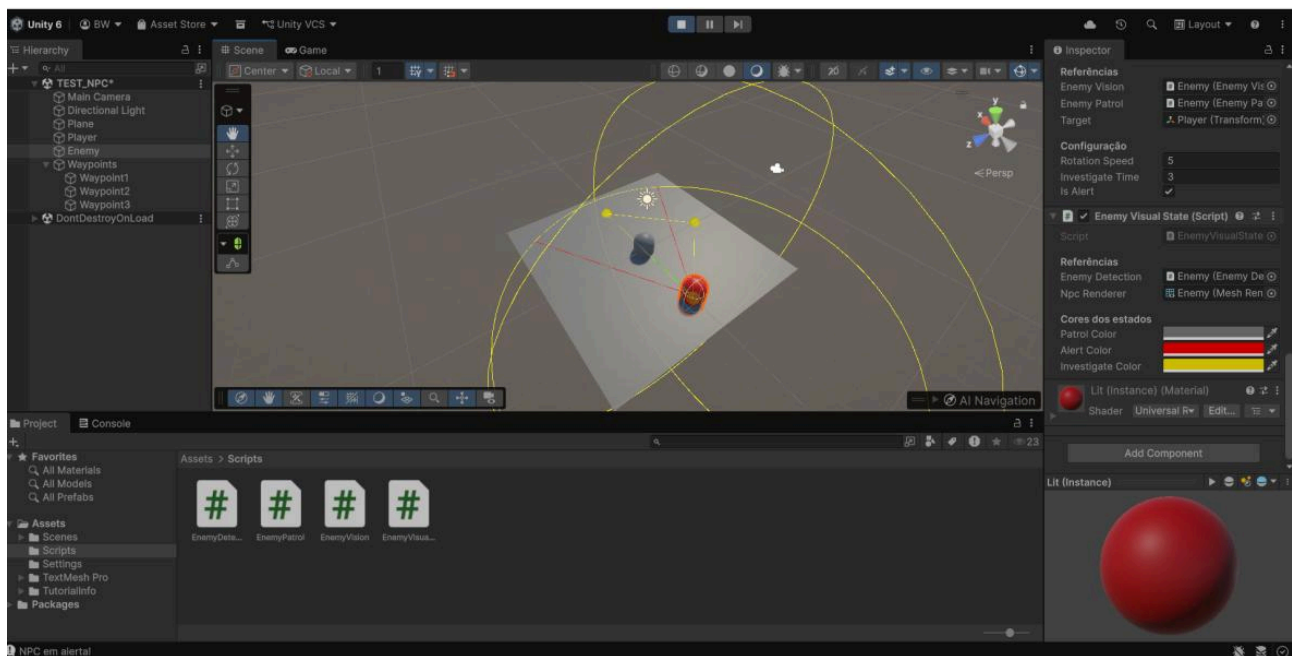
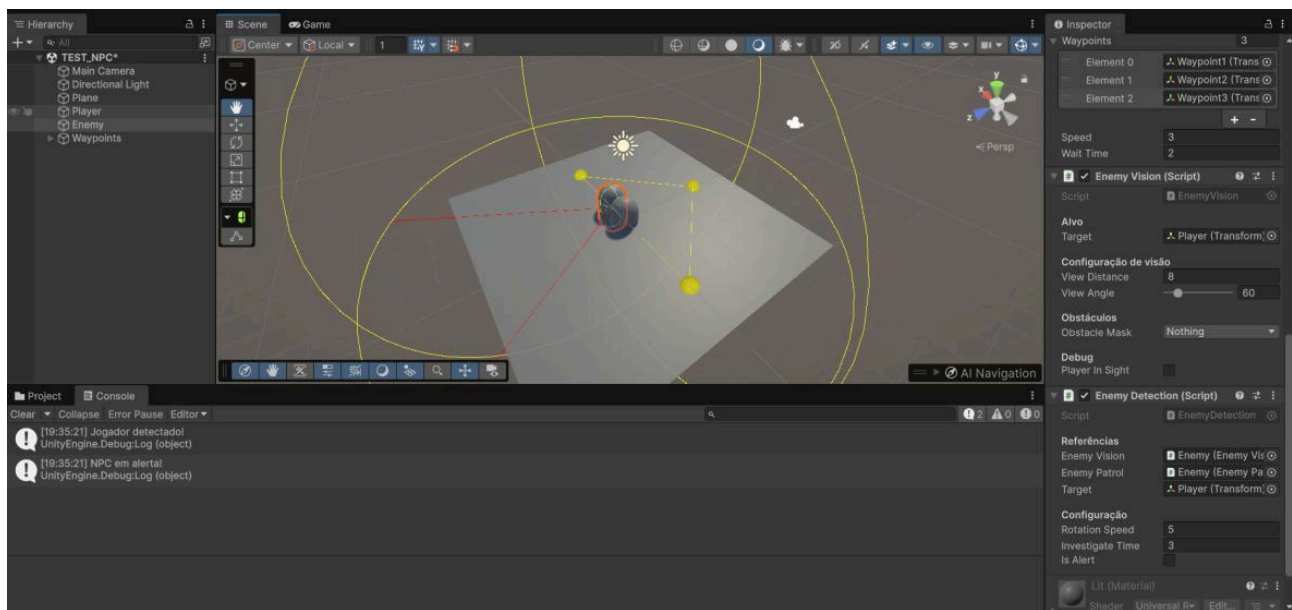
    // Raio máximo de detecção
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(origin, viewDistance);

    // Linha até o alvo com cor por estado
    if (target != null)
    {
        Gizmos.color = playerInSight ? Color.green : Color.white;
        Gizmos.DrawLine(origin, target.position);
    }
}
}

```

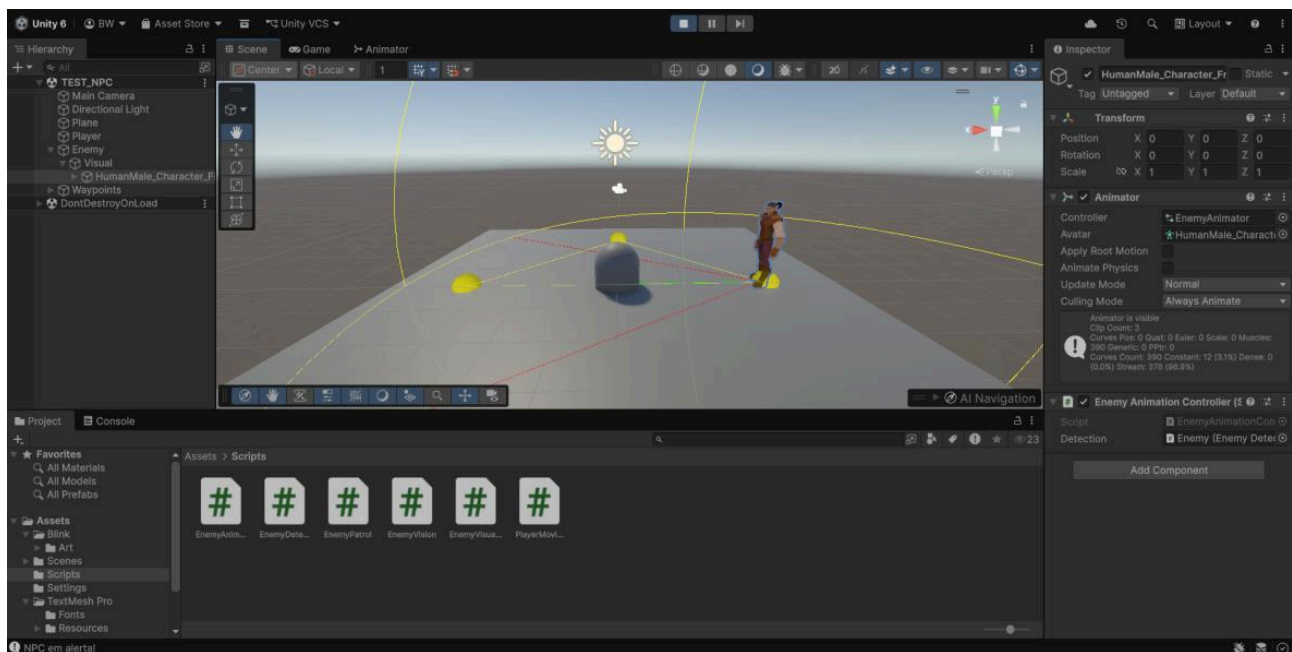
3.4.4 Sistema de Detecção e Reação do NPC (EnemyDetection.cs)

O script `EnemyDetection.cs` é responsável por processar as informações do `EnemyVision.cs` e determinar a reação do NPC ao detectar o jogador. Esta separação de responsabilidades entre “ver” e “reagir” torna a IA mais modular e fácil de manter. Quando o NPC detecta o alvo, ele interrompe a patrulha, gira para olhar para o alvo e entra em estado de alerta. O código completo do `EnemyDetection.cs` pode ser consultado no Anexo A.5.



3.5 Animação e Refinamento

A integração das animações com a lógica de IA do NPC foi um processo complexo, envolvendo a configuração do `Animator Controller` e a resolução de desafios relacionados a animações em modo somente leitura.



3.5.1 Configuração do Animator Controller

Para controlar as animações do NPC, um `Animator Controller` foi configurado. Os parâmetros `isWalking` e `isAlert` foram criados para transições entre os estados de animação. O parâmetro `isWalking` controla a transição entre os estados `Idle` (parado) e `walkForward` (caminhando), enquanto `isAlert` gerencia a transição para o estado `Alert` quando o jogador é detectado. As transições foram configuradas para garantir uma resposta imediata aos estados do NPC, desativando a opção `Has Exit Time`.

3.5.2 Desafios com Animações Read-Only e Soluções

Um desafio significativo surgiu quando se tentou adicionar `Animation Events` aos clipes de animação para sincronizar o som dos passos. As animações importadas do asset estavam marcadas como `Read-Only`, impedindo edições diretas. Diante dessa limitação, uma solução provisória foi adotada: o som dos passos passou a ser controlado por script, baseado no estado do NPC durante a patrulha, enquanto o som de alerta continuou sendo disparado no momento apropriado. Embora essa solução não ofereça sincronização exata com o contato do pé no chão, ela permitiu manter o NPC funcional e sonoramente ativo, evitando atrasos no desenvolvimento.

3.6 Sistema de Áudio

A implementação do sistema de áudio do NPC teve como objetivo adicionar feedback sonoro para os passos durante a patrulha e um som de alerta ao detectar o jogador, tornando o comportamento do NPC mais perceptível e imersivo.

3.6.1 Estrutura e Definição dos Tipos de Áudio

Para a implementação sonora, um componente `AudioSource` foi adicionado ao objeto principal do NPC, permitindo que o som acompanhasse o inimigo em sua movimentação. Essa decisão facilitou a reutilização em outras cenas e a criação de um prefab completo do inimigo. Foram definidos dois sons básicos:

- **Som de movimentação:** Utilizado para representar os passos do NPC durante a patrulha.
- **Som de alerta:** Disparado no momento em que o NPC detecta o jogador.

3.6.2 Implementação e Problemas Encontrados

Um script específico foi criado para controlar o áudio, mantendo a modularidade do sistema. No entanto, durante os testes iniciais, ocorreu um erro: `PlayOneShot was called with a null AudioClip`. Este erro indicava que o sistema estava tentando reproduzir um som sem que um arquivo de áudio estivesse atribuído ao campo correspondente no `Inspector`. A correção consistiu em atribuir manualmente os arquivos de som aos campos do componente de controle de áudio no `Inspector` do NPC. Após esse ajuste, os sons passaram a ser reproduzidos sem erro.

3.6.3 Limitação Atual do Sistema de Passos

Embora o som esteja funcionando, foi identificado que ele ainda não está sincronizado com exatidão com os passos do personagem, devido à limitação das animações `Read-only` que impediram o uso de `Animation Events`. A solução provisória, baseada em script, foi considerada funcional para a etapa atual do projeto, mas passível de refinamento futuro para uma sincronização mais precisa.

4. Testes e Resultados

Os testes foram realizados de forma iterativa ao longo do desenvolvimento, utilizando a cena `Test_NPC` para validar a lógica de IA e as mecânicas do jogador. A saída do console (`Debug.Log`) foi fundamental para depurar e verificar o comportamento dos NPCs e do jogador. Por exemplo, mensagens no console indicavam quando o jogador era detectado ou perdido de vista pelo inimigo.

Ao final do processo, o jogo “Stealth 3D” apresentou as seguintes funcionalidades:

- **Menu Inicial:** Funcional com opções de jogo, créditos e saída.
- **Cenário Noturno:** Ambiente com iluminação, skybox, neblina e pós-processamento configurados para criar uma atmosfera imersiva.
- **Movimentação do Jogador:** Controle WASD, corrida com Shift e sistema de stamina com barra de UI.
- **IA dos Inimigos:** NPCs com patrulha entre waypoints, sistema de visão com cone de detecção e bloqueio por obstáculos, e reação de alerta ao detectar o jogador.
- **Animações:** Integração de animações de caminhada e alerta, com controle via `Animator Controller`.
- **Áudio:** Sons de passos durante a patrulha e som de alerta ao detectar o jogador.

Um vídeo demonstrativo do jogo finalizado pode ser acessado através do seguinte link: <https://drive.google.com/file/d/1sVbwz24WNQGXBFGAlw0eE2Ocq0lNQUP7/view?usp=sharing>.

5. Considerações Finais

O desenvolvimento do “Stealth 3D” proporcionou aprendizados significativos sobre a integração de diferentes sistemas em um projeto de jogo na Unity. A abordagem modular para a IA dos inimigos, a configuração detalhada do ambiente e a implementação de mecânicas de jogador demonstraram a importância de um planejamento estruturado e da iteração contínua. Desafios como a integração de ferramentas de IA e a manipulação de animações `Read-Only` foram superados através de soluções adaptativas e depuração sistemática. A experiência reforçou a necessidade de uma comunicação clara e da divisão de tarefas em projetos colaborativos. O resultado final é um protótipo funcional que atende aos requisitos

iniciais e estabelece uma base sólida para futuras expansões e refinamentos, especialmente no que tange à sincronização de áudio com animações e à implementação de comportamentos de IA mais complexos.

Anexos

Anexo A: Códigos Completos

Para facilitar a leitura e manter o foco na discussão técnica no corpo principal do documento, os códigos completos dos scripts desenvolvidos para o projeto “Stealth 3D” foram compilados e estão disponíveis nesta seção.

[Anexo A.1: MenuController.cs](#) [Anexo A.2: PlayerSettings.cs](#) [Anexo A.3: EnemyPatrol.cs](#) [Anexo A.4: EnemyVision.cs](#) [Anexo A.5: EnemyDetection.cs](#)